

Terra-Pulse: Monitor Geofísico de Ressonância de Schumann e Campo Magnético

O que este dispositivo realmente mede (e o que ele não mede)

Este projeto capta **dados geofísicos reais**:
oscilações eletromagnéticas naturais
na cavidade Terra-ionosfera (Ressonância de
Schumann, ~7.83 Hz e harmônicos) e
variações do campo magnético terrestre.

O que ele NÃO faz: não mede "como a Terra está se
sentindo", não mede φ_S ,
não prova nenhuma conexão entre esses dados e
coerência humana. Ele produz um
gráfico de frequência e amplitude — a interpretação
de qualquer correlação com
HRV humana é hipótese a testar (Frente 3), não fato
assumido.

Aviso honesto sobre dificuldade real

Captar Schumann com hardware caseiro é **genuinamente difícil**, não "uma semana de projeto":

- O sinal é extremamente fraco ($\sim 0.5\text{--}1$ picotesla). A maioria dos projetos amadores bem-sucedidos usa bobinas de indução grandes (500–2000 espiras, núcleo de ferrite longo) e amplificadores de baixíssimo ruído.
- **Ruído de rede elétrica (50/60 Hz e harmônicos)** é ordens de magnitude mais forte que o sinal de Schumann e vai dominar a leitura sem filtragem agressiva e blindagem adequada. A maior parte dos "sucessos" de hobbyistas no YouTube captam principalmente ruído de rede, não Schumann real — vale ceticismo com qualquer resultado até confirmar o pico exatamente em 7.83 Hz e harmônicos (14.3, 20.8, 27.3, 33.8 Hz), não em 50/60 Hz ou seus harmônicos.
- Ambientes urbanos (o seu caso, região metropolitana do Rio) têm ruído eletromagnético muito mais alto que áreas

rurais isoladas — detectar o sinal
real ali é mais difícil ainda.

Isso não significa desistir — significa começar
sabendo que a primeira versão
provavelmente vai captar ruído, e que separar ruído
de sinal real é o próprio
trabalho científico do projeto.

Lista de Materiais (BOM)

Item	Qtd	Descrição	Custo aprox. (USD)
ESP32-S3 (ou ESP32 DevKit)	1	Microcontrolador, ADC de 12 bits	8.00
Bobina de indução ELF	1	Núcleo de ferrite, 1000+ espiras de fio esmaltado fino (ou bobina pronta de rádio AM de longa onda, reaproveitada)	15.00– 30.00
Amplificador operacional de	1	Ex: OPA2350 ou INA128	5.00

Item	Qtd	Descrição	Custo aprox. (USD)
baixo ruído		(instrumentação)	
Magnetômetro I2C	1	QMC5883L ou HMC5883L (3 eixos)	3.00
Filtro passa-baixa ativo (~40 Hz corte)	—	RC + op-amp, ou implementado digitalmente após ADC	2.00
Blindagem (gaiola de Faraday parcial)	1	Malha de cobre ou lata, para reduzir ruído de RF ambiente	5.00
Cabo blindado (coax fino)	—	Da bobina até o amplificador, para reduzir captação de ruído	3.00
Total			~40–55 USD

Esquema de Sinal (Diagrama de Blocos)

[Bobina ELF] → [Pré-amplificador baixo ruído] → [Filtro passa-baixa ~40Hz]

↓

[ADC do ESP32 (12-bit, ~200 Hz sample rate)]

↓

[FFT no ESP32 ou envio raw ao PC]

↓

[Detecção de picos em 7.83/14.3/20.8 Hz]

[Magnetômetro I2C] → [ESP32 via I2C] →
[Cálculo de variância/desvio-padrão do campo]

Código Arduino/ESP32 (honesto, sem narrativa mística)

```
/*  
 * Terra-Pulse: Monitor de Ressonância de  
 * Schumann e Campo Magnético
```

```

* -----
-----
* Este código faz DUAS coisas, sem
interpretação além do dado bruto:
* 1. Amostra o sinal ELF via ADC e
calcula um espectro de frequência (FFT)
* 2. Lê o magnetômetro e calcula
variabilidade (desvio-padrão) ao longo do
tempo
*
* NENHUMA linha aqui interpreta os dados
como "consciência", "phi_S" ou
* "sentimento planetário". A saída é:
frequência, amplitude, variância.
*/

#include <Wire.h>
#include <arduinoFFT.h>

// --- Configuração ADC / FFT ---
#define SAMPLES 256 //
Potência de 2, para FFT
#define SAMPLING_FREQUENCY 200 // Hz
(Nyquist cobre até 100 Hz, suficiente para
Schumann)
#define ADC_PIN 34 // Pino
ADC do ESP32 conectado ao pré-amplificador

double vReal[SAMPLES];
double vImag[SAMPLES];
ArduinoFFT<double> FFT = ArduinoFFT<double>
(vReal, vImag, SAMPLES,
SAMPLING_FREQUENCY);

```

```

// --- Configuração Magnetômetro (QMC5883L
via I2C) ---
#define MAG_ADDR 0x0D
float mag_x_history[100];
int mag_history_idx = 0;

void setup() {
    Serial.begin(115200);
    Wire.begin();
    setupMagnetometer();

    Serial.println("timestamp_ms,freq_peak_hz,f
req_peak_amplitude,mag_x,mag_y,mag_z,mag_va
riance");
}

void loop() {
    // --- 1. Amostragem ELF para FFT ---
    unsigned long sampling_period_us =
round(1000000.0 / SAMPLING_FREQUENCY);
    for (int i = 0; i < SAMPLES; i++) {
        unsigned long t0 = micros();
        vReal[i] = analogRead(ADC_PIN);
        vImag[i] = 0;
        while (micros() - t0 <
sampling_period_us) { /* aguarda próximo
sample */ }
    }

    FFT.windowing(vReal, SAMPLES,
FFT_WIN_TYP_HAMMING, FFT_FORWARD);
    FFT.compute(vReal, vImag, SAMPLES,
FFT_FORWARD);
    FFT.complexToMagnitude(vReal, vImag,

```

```

SAMPLES);

    double peakFreq = FFT.majorPeak(vReal,
SAMPLES, SAMPLING_FREQUENCY);
    double peakAmplitude =
getAmplitudeAtFreq(vReal, peakFreq,
SAMPLING_FREQUENCY, SAMPLES);

    // --- 2. Leitura do magnetômetro ---
    float mx, my, mz;
    readMagnetometer(mx, my, mz);
    mag_x_history[mag_history_idx % 100] =
mx;
    mag_history_idx++;
    float mag_variance =
computeVariance(mag_x_history,
min(mag_history_idx, 100));

    // --- 3. Saída: SÓ DADO, sem narrativa -
--
    Serial.print(millis());
Serial.print(",");
    Serial.print(peakFreq, 3);
Serial.print(",");
    Serial.print(peakAmplitude, 3);
Serial.print(",");
    Serial.print(mx, 3); Serial.print(",");
    Serial.print(my, 3); Serial.print(",");
    Serial.print(mz, 3); Serial.print(",");
    Serial.println(mag_variance, 5);

    delay(1000); // 1 leitura por segundo
}

```



```

double getAmplitudeAtFreq(double*
magnitudes, double targetFreq, int
sampleRate, int samples) {
    int bin = round(targetFreq * samples /
sampleRate);
    if (bin < 0 || bin >= samples / 2) return
0;
    return magnitudes[bin];
}

float computeVariance(float* data, int n) {
    if (n < 2) return 0;
    float mean = 0;
    for (int i = 0; i < n; i++) mean +=
data[i];
    mean /= n;
    float var = 0;
    for (int i = 0; i < n; i++) var +=
(data[i] - mean) * (data[i] - mean);
    return var / (n - 1);
}

void setupMagnetometer() {
    Wire.beginTransmission(MAG_ADDR);
    Wire.write(0x0B); Wire.write(0x01); //
Reset
    Wire.endTransmission();
    Wire.beginTransmission(MAG_ADDR);
    Wire.write(0x09); Wire.write(0x1D); //
Modo contínuo, 200Hz, escala 8G
    Wire.endTransmission();
}

void readMagnetometer(float &x, float &y,

```

```
float &z) {  
    Wire.beginTransaction(MAG_ADDR);  
    Wire.write(0x00);  
    Wire.endTransmission();  
    Wire.requestFrom(MAG_ADDR, 6);  
    if (Wire.available() == 6) {  
        int16_t rawX = Wire.read() |  
(Wire.read() << 8);  
        int16_t rawY = Wire.read() |  
(Wire.read() << 8);  
        int16_t rawZ = Wire.read() |  
(Wire.read() << 8);  
        x = rawX / 12000.0; // conversão para  
        Gauss (aproximada, calibrar depois)  
        y = rawY / 12000.0;  
        z = rawZ / 12000.0;  
    }  
}
```

Validação obrigatória antes de confiar em qualquer leitura

1. Confirme o pico em 7.83 Hz especificamente, não em qualquer frequência baixa aleatória. Se o pico dominante estiver em 50/60 Hz ou seus harmônicos (100, 120, 150, 180 Hz), isso é ruído de rede elétrica, não Schumann.

2. **Teste em modo avião / sem eletrônicos próximos** para reduzir interferência de RF local antes de qualquer leitura "real".
 3. **Compare leituras em horários e locais diferentes.** Ressonância de Schumann varia naturalmente com hora do dia (atividade de raios global) — isso é esperado e documentado na literatura geofísica, não é anomalia.
 4. **Registre tudo com timestamp** — isso vira o dataset bruto pro teste de correlação da Frente 3, sem nenhuma interpretação embutida no dado.
-

Próximo passo sugerido

Depois de ter esse hardware rodando e confirmando um pico real em 7.83 Hz (o que pode levar semanas de calibração, não dias), aí sim faz sentido avançar pra Frente 3 — comparar esse dataset bruto com HRV de um grupo, como teste cego real, sem presumir o resultado.